

Software safety assessment based on a subordinated Markov chain

Hiroyuki Okamura · Tadashi Dohi

Received: 24 October 2010 / Revised: 27 December 2010 / Published online: 13 April 2011

© The Society for Reliability Engineering, Quality and Operations Management (SREQOM), India and The Division of Operation and Maintenance, Lulea University of Technology, Sweden 2011

Abstract Software safety can be defined as the probability that any software hazard does not occur during a pre-specified time period. This paper proposes a new software safety assessment model based on a subordinated Markov chain, which consists of both non-homogeneous Poisson process (NHPP) and discrete time Markov chain (DTMC). In the model, NHPP and DTMC indicate the cumulative number of software errors experienced in testing phase and their impact, respectively. Using the data set of three types of software errors (critical, major and minor errors) collected in an actual software development project, we examine the effectiveness of the proposed model by compared with two existing models. Through the experiment, it is shown that the proposed model is superior to the existing ones in terms of fitting ability.

Keywords Software safety · Non-homogeneous Poisson process · Subordinated Markov chain

1 Introduction

During the last four decades, software reliability models (SRMs) have been extensively developed by many authors (Goel and Okumoto 1979; Lyu 1996; Musa and Okumoto 1984; Musa 1999; Schick and Wolverton 1978 and Yamada

and Osaki 1985). In fact, more than 200 SRMs have been proposed to assess the software reliability. The most traditional and important class of SRMs is the non-homogeneous Poisson process (NHPP) model by Goel and Okumoto (1979), and a number of variants have been proposed by Yamada and Osaki (1985), Musa and Okumoto (1984) and others. Each of these NHPP-based SRMs is based on an inherent debugging scenario, but almost all models represent the reliability growth phenomenon that is caused by debugging the faults detected in testing phase. In the SRMs, we focus on the event that any software fault is detected. However, all the faults do not always cause a catastrophic software failure, i.e., software hazard.

Software safety can be defined as the probability that the software hazard caused by critical errors does not occur during a pre-specified time period (Keene 1992 and Leveson 1986). Since our lives are being controlled by many safety-critical and mission-critical software systems, the concept of software safety becomes recognized as one of the most important topics in the software engineering. Keene (1992) introduced the so-called fault-tree approach to identify the software hazard. Tokuno and Yamada (1999) developed a simple continuous-time Markov model to assess both the software safety and software reliability. In the software safety, it is important to distinguish software failures according to their impact on the software system. Needless to say, the impact of a critical software error is much bigger than that of a minor error such as misspelling a message. Misra (1983) and Pham (2000) analyzed the quantitative software safety/reliability based on the data set of software errors classified by their impacts. However, their models used a simple superposition of independent NHPP-based SRMs, and thus their assumption is quite strong on the software safety assessment.

H. Okamura (✉) · T. Dohi
Department of Information Engineering, Graduate School
of Engineering, Hiroshima University, 1-4-1 Kagamiyama,
Higashi-Hiroshima 739-8527, Japan
e-mail: okamu@rel.hiroshima-u.ac.jp

T. Dohi
e-mail: dohi@rel.hiroshima-u.ac.jp

This paper proposes a new software safety assessment model based on a subordinated Markov chain, which consists of both NHPP and discrete time Markov chain (DTMC). In our model, NHPP and DTMC indicate the cumulative number of errors experienced in testing phase and their impacts, respectively. The essential difference between our model and the existing models is that the type of detected error is dominated by the type of previous, i.e., we assume that the type of errors has a Markovian property. Since it is quite difficult to identify and model the correlation between types of software errors, our approach may be validated in terms of technical convenience.

We also develop an iterative algorithm based on the EM (expectation–maximization) principle (Dempster et al. 1973 and Wu 1983) to obtain the maximum likelihood estimates for our model parameters. Using the data set of three types of software errors (critical, major and minor errors) observed in an actual software development project, the proposed model is examined to validate the goodness-of-fit and prediction performance by compared with two existing superposed NHPP-based SRMs. In the goodness-of-fit test, we use three kinds of criteria; Kolmogorov–Smirnov distance, mean square error and Akaike’s information criterion (Akaike 1973). Finally, the paper summarizes the results and is concluded with some remarks.

2 Software safety assessment

2.1 Software reliability and safety

In this paper, software reliability and safety are defined as follows:

Software Reliability probability that any software error does not occur during a specified time period.

Software Safety probability that the software error causing a software hazard does not occur during a specified time period.

2.2 Software reliability models

NHPP-based SRMs are widely used to assess the software reliability from empirical data observed in testing phase of software development. Let $\{N(t); t \geq 0\}$ be a stochastic process meaning the cumulative number of software errors experienced before time t . Suppose that $N(t)$ has the following properties:

- (A-1) $N(0) = 0$,
- (A-2) $\{N(t); t \geq 0\}$ has independent increments,
- (A-3) $\Pr\{N(t + \Delta t) - N(t) = 1\} = \lambda(t)\Delta t + o(\Delta t)$,
- (A-4) $\Pr\{N(t + \Delta t) - N(t) \geq 2\} = o(\Delta t)$,

where $\lambda(t)$ denotes the failure intensity function and $o(\Delta t)$ is the second or higher order term of Δt . When $N(t)$ has the properties (A-1)–(A-4), the counting process $N(t)$ is an NHPP and its probability mass function (p.m.f.) is given by

$$\Pr\{N(t) = n\} = \frac{\Lambda(t)^n}{n!} \exp(-\Lambda(t)), t \geq 0, n = 0, 1, \dots, \quad (1)$$

where $\Lambda(t) = \int_0^t \lambda(s)ds$, is called the mean value function and represents the expected cumulative number of software errors experienced before time t .

All of existing NHPP-based SRMs are characterized by mean value functions. Typical NHPP-based SRMs have the following mean value functions:

1. Exponential (Exp) Model (Goel and Okumoto 1979):

$$\Lambda(t) = a\{1 - \exp(-bt)\}, \quad (2)$$

where $a(>0)$ is the expected total number of inherent errors, $b(>0)$ is the error occurrence rate per unit time. 2. Gamma (Gam) Model (Yamada and Osaki 1985):

$$\Lambda(t) = a\{1 - (1 + bt)\exp(-bt)\} \quad (3)$$

3. Rayleigh (Ral) Model (Schick and Wolverton 1978):

$$\Lambda(t) = a\{1 - \exp(-bt^2)\} \quad (4)$$

4. Logarithmic Poisson Execution Time (LPe) Model (Musa and Okumoto 1984):

$$\Lambda(t) = \frac{1}{\theta} \ln(\lambda_0 \theta t + 1), \quad (5)$$

where $\lambda_0(>0)$ is the initial failure intensity, $\theta(>0)$ is the failure intensity decay parameter. The mean value functions of (1), (2) and (3) involve the parameter of the number of inherent faults a . Such SRMs are according to the common assumption that the number of inherent errors is finite, i.e.,

$$\lim_{t \rightarrow \infty} \Lambda(t) = a < \infty. \quad (6)$$

On the other hand, the mean value function of (iv) has the following property:

$$\lim_{t \rightarrow \infty} \Lambda(t) \rightarrow \infty. \quad (7)$$

This implies that the number of inherent errors is infinite. According to the properties, all the NHPP-based SRMs are classified into finite error model and infinite

error model, which are often called NHPP-I and NHPP-II models, respectively.

2.3 Modeling of impact of software errors

In the software safety assessment, software errors should be classified into several types in terms of their impacts on the system. For example, we define the type; *critical*, as software errors cause the software hazard. As mentioned before, the software safety is defined as the probability that the software hazard does not occur during a specified time period. According to the classification of software errors, the software safety assessment is reduced into assessing the software reliability only for the critical errors.

Suppose that all the software errors can be classified to $q(\geq 1)$ types, which are labeled as $1, \dots, q$ in the order of lower impact, i.e., the impact of type i error is bigger than that of type $i - 1$. Let $R_i(t|s)$ denote the probability that the type i error does not occur during the time period $(s, s + t]$. If the occurrence of the type i errors follows the NHPP with the mean value function $\Lambda_i(t)$, the probability $R_i(t|s)$ is given by

$$R_i(t|s) = \exp\{-(\Lambda_i(s+t) - \Lambda_i(s))\}, \quad (8)$$

where $\Lambda_i(t)$ means the expected cumulative number of type i errors by time t .

The software hazard would be caused if and only if the type q error occurs. Then the software safety $S_q(t|s)$ can be defined by the probability that the type q error does not occur during the time period $(s, s + t]$. In fact, Misra (1983) considered the same problem based on three types of software errors; critical error, major error and minor error, and assessed the software reliability by using a superposition of three independent NHPP-based SRMs. In his modeling framework, assuming that the software hazard is induced by only the critical error, the software safety is given by

$$S_q(t|s) = R_q(t|s). \quad (9)$$

Here we consider a slightly extended model from Misra (1983) by being applied to several error types. If the occurrence of type i errors follows the NHPPs with the mean value function $\Lambda_i(t)$ and if the types $(k, \dots, q)$ errors induce the software hazard, then the software safety in the superposed NHPP-based SRM can be obtained by

$$S_k(t|s) = \prod_{i=k}^q R_i(t|s). \quad (10)$$

The above model by Misra (1983) arises practical problems. For instance, when we wish to estimate the accurate software safety from empirical data, many

observations for the software errors causing the software hazard are needed. In particular, since Misra's model consists of the independent NHPPs, we cannot use empirical data for minor and major errors for estimating the parameters of NHPP-base SRMs of critical errors. In other words, Misra's model requires many observations for critical errors to derive the software safety accurately. However, it should be noted that the critical errors causing the software hazard rarely are detected in testing phase, and therefore it is not easy to obtain the sufficient data for critical errors.

Pham (2000) proposed just a little bit different parameter estimation method by the similar modeling framework to Misra (1983). He modified the parameters on the expected total number of inherent faults as follows.

$$a_i = ap_i, \quad i = 1, \dots, q, \quad (11)$$

where the proportion of the type i errors, $p_i \in (0,1)$, satisfies $\sum_{i=1}^q p_i = 1$. Although this model is essentially same as Misra (1983), the estimation method of the model parameter was improved. More precisely, Pham (2000) proposed the estimation method where the arithmetic rate of type i errors is substituted to p_i . This method can reduce the number of model parameters essentially. However, Pham's method is available at only the NHPP-I models assuming the finite number of inherent faults.

This paper develops a new software safety/reliability model to deal with multiple types of software errors, where each type of software errors occurs under a milder condition than the independence, i.e., Markovian property.

3 Subordinated markov chain model

3.1 Model description

Our model makes the following assumptions:

- (B-1) the software errors occur in accordance with an NHPP.
- (B-2) the type of an error depends on the type of the previously detected error.

The assumption (B-2) means that the transition of error type follows a DTMC. This is the most different point from Misra (1983) and Pham (2000). It may not be so easy to answer the question why the Markovian property should be assumed in modeling the software error types. However, as well known, the Markov property can provide many technical advantages. That is, noting that the Markovian assumption is rather weaker than independence, this assumption may be validated in terms of technical benefit.

Let $\{N(t); t \geq 0\}$ be the cumulative number of software errors experienced before time t and $\{X_n, n = 1, 2, \dots\}$ be the type of the n -th occurrence error. According to (B-1), the number of all types of errors experienced before time t follows an NHPP with the mean value function:

$$\Lambda(t) = \sum_{i=1}^q \Lambda_i(t), \quad (12)$$

where $\Lambda_i(t)$ is the expected cumulative number of type i errors before time t . On the other hand, according to (B-2), the sequence $\{X_n, n = 1, 2, \dots\}$ can be modeled by a DTMC with state space $N = \{1, 2, \dots, q\}$. Let $\mathbf{P}$ denote the transition probability matrix of the DTMC. Since the NHPP is assumed to be independent of the DTMC, we have the joint probability that n errors occur before time t and that the type of the last error is j , as follows.

$$\begin{aligned} \Pr\{N(t) = n, X_{N(t)} = j | N(0) = 0, X_0 = i\} \\ = \Pr\{N(t) = n | N(0) = 0\} \Pr\{X_n = j | X_0 = i\} \\ = \frac{\Lambda(t)^n}{n!} \exp(-\Lambda(t)) [\mathbf{P}^n]_{ij} \end{aligned} \quad (13)$$

where $[\mathbf{P}^n]_{ij}$ is the (i, j) -element of the matrix $\mathbf{P}^n$. The stochastic process $\{X_{N(t)}, t \geq 0\}$ is called a subordinated Markov chain, on which the transition for the DTMC occurs in accordance with the NHPP (see e.g. Kijima (1997)). Let $\boldsymbol{\pi}(t) = (\Lambda_1(t), \dots, \Lambda_q(t))$ be the row vector which represents the expected cumulative number of errors before time t . Then we have

$$\boldsymbol{\pi}(t) = \mathbf{x}_0 \sum_{n=0}^{\infty} \sum_{m=0}^n \mathbf{P}^m \frac{\Lambda(t)^n}{n!} \exp(-\Lambda(t)), \quad (14)$$

where $\mathbf{x}_0$ is the initial probability vector. The expected number of software errors $\boldsymbol{\pi}(t)$ can be alternatively computed by the recursive formula:

$$\boldsymbol{\pi}(t) = \mathbf{x}_0 \sum_{n=0}^{\infty} \mathbf{A}_n, \quad (15)$$

$$\mathbf{A}_0 = \mathbf{O}, \quad (16)$$

$$\mathbf{A}_1 = \Lambda(t) \exp\{-\Lambda(t)\} \mathbf{P}, \quad (17)$$

$$\mathbf{A}_n = \frac{\Lambda(t)}{n} \mathbf{A}_{n-1} + \mathbf{B}_n, \quad (18)$$

$$\mathbf{B}_0 = \exp\{-\Lambda(t)\} \mathbf{I}, \quad (19)$$

$$\mathbf{B}_n = \frac{\Lambda(t)}{n} \mathbf{P} \mathbf{B}_{n-1} \quad (20)$$

where $\mathbf{O}$ and $\mathbf{I}$ are the zero and identity matrices, respectively.

Next, we derive the quantitative software safety. When the types $(k, \dots, q)$ errors are assumed to cause a software hazard, the software safety is given by

$$\begin{aligned} S_k(t|s) &= \bar{\mathbf{x}}_k \sum_{n=0}^{\infty} \left\{ \frac{\{-(\Lambda(s+t) - \Lambda(s))\}^n}{n!} \right. \\ &\quad \times \exp\{-(\Lambda(s+t) - \Lambda(s))\} \bar{\mathbf{P}}_k^n \Big\} \mathbf{e} \\ &= \bar{\mathbf{x}}_k \exp\{-(\Lambda(s+t) - \Lambda(s))\} (\mathbf{I} - \bar{\mathbf{P}}_k) \mathbf{e}, \end{aligned} \quad (21)$$

where $\mathbf{e}$ is the column vector whose elements are all 1. The vector $\bar{\mathbf{x}}_k$ and the matrix $\bar{\mathbf{P}}_i$ are the subvector of $\mathbf{x}_0$ and the submatrix of $\mathbf{P}$, respectively, in the following:

$$\bar{\mathbf{x}}_k = (x_1, \dots, x_{k-1}), \quad (22)$$

$$\bar{\mathbf{P}}_i = \begin{pmatrix} P_{1,1} & \cdots & P_{1,k-1} \\ \vdots & \ddots & \vdots \\ P_{k-1,1} & \cdots & P_{k-1,k-1} \end{pmatrix}, \quad (23)$$

where x_i and p_{ij} are the i -element of $\mathbf{x}_0$ and the (i, j) -element of $\mathbf{P}$, respectively. As a special case, when all the types of error induce the software hazard, Eq. 21 is rewritten as

$$S_1(t|s) = \exp\{-(\Lambda(s+t) - \Lambda(s))\} \quad (24)$$

which is equivalent to the software reliability based on the NHPP-based SRMs (Lyu 1996).

3.2 Parameter estimation

We describe the parameter estimation for the subordinated Markov chain model. The observations of software errors are given by $\mathcal{D}(1), \dots, \mathcal{D}(m)$, where $\mathcal{D}(k) = (t(k), n_1(k), \dots, n_q(k))$, $k = 1, 2, \dots, m$, consists of the testing time $t(k)$, and the number of the type i errors $n_i(k)$ ($i = 1, \dots, q$) observed during unit time period such as 1 day, 1 week and 1 month.

The NHPP which represents the cumulative number of software errors is independent of the DTMC denoting the impact of errors. Thus the maximum likelihood estimation for the subordinated Markov chain model can be divided into the estimation procedures for NHPP and DTMC parameters.

Let $\Lambda(t; \boldsymbol{\theta})$ be the mean value function of an NHPP, where $\boldsymbol{\theta}$ is a parameter set. For the set of observed data $\mathcal{D}(1), \dots, \mathcal{D}(m)$, the model parameters are determined so as to maximize the log-likelihood function of the NHPP:

$$\begin{aligned} \log L_{\text{NHPP}}(\boldsymbol{\theta} | \mathcal{D}(1), \dots, \mathcal{D}(m)) \\ = \sum_{i=1}^m n(i) \log(\Lambda(s(i); \boldsymbol{\theta}) - \Lambda(s(i-1); \boldsymbol{\theta})) \\ - \Lambda(s(m); \boldsymbol{\theta}) - \sum_{i=1}^m \log n(i)!, \end{aligned} \quad (25)$$

where $n(k) = \sum_{i=1}^q n_i(k)$, $s(0) = 0$ and $s(k) = \sum_{i=1}^k t(i)$. In general, the above log-likelihood function is a non-linear

function. Although it is not easy to find the parameters which maximize the log-likelihood function, several iterative algorithms are developed to estimate the parameters for the SRMs (Okamura et al. 2002, 2003, 2004; Okamura and Dohi 2006a, b, 2008; Ohishi et al. 2009). By using such existing estimation methods, we can get the estimates of model parameters for the NHPP.

Secondly we consider parameter estimation of the DTMC. Generally speaking, it is easy to find the maximum likelihood estimates of the DTMC if we completely observe transitions of types of errors. Let p_{ij} denote the (i, j) -element of the matrix $\mathbf{P}$. Given the history of transitions of the DTMC, we can derive each of estimates of the transition probabilities as the ratio of the number of corresponding transitions over the total number of transitions. Then we obtain

$$\hat{p}_{ij} = \frac{N_{ij}}{N_j}, \quad (26)$$

where N_{ij} is the number of transitions from i to j and $N_j = \sum_i N_{ij}$ is the total number of transitions. However, in the case where the observations form grouped data, the estimation of DTMC is much harder than the above case. The grouped data means that the detailed transition behavior of the DTMC is not observed. In our modeling, since the error data consists of only the number of the type i errors during unit time period, we may use the grouped data. Although the likelihood function for the DTMC can be also formulated under the grouped data $\mathcal{D}(1), \dots, \mathcal{D}(m)$, some numerical techniques are required to computing the maximum likelihood estimates. This paper proposes an estimation procedure for the DTMC under the grouped data by using the EM (expectation–maximization) principle (Dempster et al. 1973 and Wu 1983).

Define the following recursive formula:

$$\xi_{ij}(n_1, \dots, n_q; \mathbf{v}) = \Delta_{n_j} \xi_{ij}(n_1, \dots, n_q; \mathbf{v}) \mathbf{P} + \Delta_{n_j} \psi(n_1, \dots, n_q; \mathbf{v}) \tilde{\mathbf{P}}(i, j), \quad (27)$$

$$\xi_{ij}(0, \dots, 0; \mathbf{v}) = (\mathbf{0}, \dots, \mathbf{0}), \quad \text{for } i, j = 1, \dots, q \quad (28)$$

$$\begin{aligned} \psi(n_1, \dots, n_q; \mathbf{v}) &= (\Delta_{n_1} \psi(n_1, \dots, n_q; \mathbf{v}) \boldsymbol{\mu}_1, \\ &\quad \Delta_{n_2} \psi(n_1, \dots, n_q; \mathbf{v}) \boldsymbol{\mu}_2, \\ &\quad \vdots \\ &\quad \Delta_{n_q} \psi(n_1, \dots, n_q; \mathbf{v}) \boldsymbol{\mu}_q), \end{aligned} \quad (29)$$

$$\psi(0, \dots, 0; \mathbf{v}) = \mathbf{v}, \quad (30)$$

where Δ_{n_i} is the shift operator, i.e., for any function f ,

$$\begin{aligned} \Delta_{n_i} f(n_1, \dots, n_i, \dots, n_q) \\ = \begin{cases} f(n_1, \dots, n_i - 1, \dots, n_q) & \text{for } n_i > 0, \\ 0 & \text{for } n_i = 0. \end{cases} \end{aligned} \quad (31)$$

In the recursive formula, $\tilde{\mathbf{P}}(i, j)$ is the $q \times q$ matrix where only the (i, j) -element is p_{ij} and the other elements are all 0. The column vector $\boldsymbol{\mu}_i$ indicates the i -th column of $\mathbf{P}$. Let $\mathbf{M}_m$ and $\mathbf{v}$ denote the state in the DTMC after m transitions and the initial probability vector, respectively. Then the vector $\xi_{ij}(n_1, \dots, n_q; \mathbf{v})$ is defined by the expected number of transitions from i to j under the observations (grouped data) $n_1, \dots, n_q$, whose k -element can be represented as

$$[\xi_{ij}(n_1, \dots, n_q; \mathbf{v})]_k = \sum_{l=0}^{m-1} \Pr\{M_l = i, M_{l+1} = j\} \Pr\{M_m = k\}, \quad (32)$$

where M_0 is the initial state, and its probability is given by $\Pr\{M_0 = k\} = [\mathbf{v}]_k$. Also, the k -element of $\psi(n_1, \dots, n_q; \mathbf{v})$ is given by $\Pr\{M_m = k\}$. By using the expected number of transitions for unit time period $\xi_{ij}(\cdot)$, the EM algorithm for the data set $\mathcal{D}(1), \dots, \mathcal{D}(m)$ can be obtained as follows.

(E-step): For all i, j , we calculate the following equations:

$$\mathbf{x}(1) = \psi(n_1(1), \dots, n_q(1); x_0) \quad (33)$$

$$\mathbf{n}_{ij}(1) = \xi_{ij}(n_1(1), \dots, n_q(1); x_0) \quad (34)$$

$$\mathbf{x}(k) = \psi(n_1(k), \dots, n_q(k); \mathbf{x}(k-1)) \quad (35)$$

$$\mathbf{n}_{ij}(k) = \mathbf{n}_{ij}(k-1) + \xi_{ij}(n_1(k), \dots, n_q(k); \mathbf{x}(k-1)) \quad (36)$$

$$\mathbf{x}(m) = \psi(n_1(m), \dots, n_q(m); \mathbf{x}(m-1)) \quad (37)$$

$$\mathbf{n}_{ij}(m) = \mathbf{n}_{ij}(m-1) + \xi_{ij}(n_1(m), \dots, n_q(m); \mathbf{x}(m-1)) \quad (38)$$

(M-step): Update the estimates by

$$\hat{p}_{ij} = \frac{n_{ij}(m)e}{\sum_{j=1}^q n_{ij}(m)e} \quad \text{for all } i, j. \quad (39)$$

By updating the estimates according to the above algorithm repeatedly, the maximum likelihood estimates for the DTMC can be derived numerically. Based on this estimation algorithm, we can obtain the estimates satisfying $0 < p_{ij} < 1$. However, it may not be useful in the case where the number of software errors is extremely large.

4 Real data analysis

We present the data analysis with the subordinated Markov chain model. The data is collected from the project for the Shuttle Ground System (Misra 1983) and consists of weekly summaries, namely, grouped data for every week. The total testing period is 38 weeks (=2456.9 h). After detecting the software fault causing the software error, the

number of errors and their types (critical, major and minor) are reported every week. For brevity, the independent NHPP model in Misra (1983), the modified NHPP model in Pham (2000) and the subordinated Markov chain model proposed in the paper are called Model 1, Model 2 and Model 3, respectively.

The model parameters are estimated by the maximum likelihood estimation method described in the previous section. In particular, the estimation method for the NHPP-I (Exp, Gam, Ral) proposed in Okamura et al. (2002, 2003) are applied to estimating model parameters. Table 1 presents the estimated parameters after the 38-th week testing, i.e., we use all the observation data. In these results, we run 1,000 times iteration of the EM algorithm to compute the maximum likelihood estimates, where the initial parameters are given by $a = 1.0$ and $b = 1.0$. Since the LPe belongs to the NHPP-II model, we use the Newton's method to obtain the estimates of model parameters. In general, the non-linear equation can be solved by the Newton's method. However, it is not always effective in the estimation of model parameters, because the resulting estimates often take invalid values such as negative values (see Table 1).

On the other hand, the maximum likelihood estimate of the DTMC at the 38-th week in Model 3 is given by

$$\hat{P} = \begin{pmatrix} 0.0000 & 0.0000 & 1.0000 \\ 0.0000 & 0.3281 & 0.6719 \\ 0.0267 & 0.3586 & 0.6147 \end{pmatrix} \quad (40)$$

From the above result, it can be seen that the critical software errors seldom occur after the occurrence of a critical error or a major error.

4.1 Goodness-of-fit test

We investigate the goodness-of-fit for the observed software error data. In this paper, three kinds of criteria are used: Kolmogorov–Smirnov (KS) distance, mean square error (MSE) and Akaike's information criteria (AIC).

The KS distance for the type i error is defined as follows.

$$D_i = \max_{1 \leq k \leq m} \{D_i(k)\},$$

$$D_i(k) = \max \left\{ \left| \frac{\Lambda_i(s(k))}{\Lambda_i(s(m))} - \frac{\sum_{l=0}^k n_i(l)}{\sum_{l=0}^m n_i(l)} \right|, \left| \frac{\Lambda_i(s(k))}{\Lambda_i(sm)} - \frac{\sum_{l=0}^{k-1} n_i(l)}{\sum_{l=0}^m n_i(l)} \right| \right\}. \quad (41)$$

This implies the maximum distance from the expected number of errors in the parametric model and can be regarded as non-parametric estimates for the number of errors. The KS distance is often used to test statistically whether the probability model fits to the data or not.

Table 1 Parameter estimation after the 38-th week testing

(a) Model 1			
NHPP	(a ₁ , b ₁)	(a ₂ , b ₂)	(a ₃ , b ₃)
Exp	(146.24, 1.13e−05)	(166.16, 2.63e−04)	(412.64, 1.81e−04)
Gam	(25.46, 2.86e−04)	(90.97, 1.44e−03)	(176.60, 1.33e−03)
Ral	(40.66, 1.72e−08)	(82.28, 5.34e−07)	(161.45, 4.12e−07)
LPe	(λ ₀ : 8.95e−04, θ: −3.37e−01)	(λ ₀ : 4.70e−02, θ: 9.07e−03)	(λ ₀ : 8.92e−02, θ: 5.00e−03)
(b) Model 2			
NHPP	(a ₁ , b ₁)	(a ₂ , b ₂)	(a ₃ , b ₃)
Exp	(8.22, 2.29e−04)	(162.32, 2.75e−04)	(304.10, 2.70e−04)
Gam	(4.60, 9.35e−04)	(90.79, 1.52e−03)	(170.09, 1.44e−03)
Ral	(4.24, 2.23e−07)	(83.64, 5.71e−07)	(156.70, 4.59e−07)
(c) Model 3			
NHPP	(a, b)		
Exp	(607.46, 1.95e−04)		
Gam	(273.83, 1.35e−03)		
Ral	(248.18, 4.42e−07)		
LPe	(λ ₀ : 1.36e−01, θ: 3.03e−03)		

The second criterion is the MSE, which is defined by

$$\text{MSE} = \sum_{k=1}^m \sum_{i=1}^q \frac{\left(\Lambda_i(s(k)) - \sum_{l=1}^k n_i(l) \right)^2}{m}. \quad (42)$$

This criterion indicates the square error between Λ_i and the number of observed errors, and is similar to the KS distance in the comparison of the expected number of errors.

The AIC is also used as the criteria of goodness-of-fit. The AIC is based on the Kullback–Leibler information (Kullback 1959) and the asymptotic normality of the maximum likelihood estimator, which is often used to investigate the goodness of the probability models estimated by the maximum likelihood estimation. Formally, the AIC is defined as follows.

$$\begin{aligned} \text{AIC} = & -2(\text{the maximum logarithmic likelihood}) \\ & + 2(\text{the degree of model freedom}), \end{aligned} \quad (43)$$

where n is the total number of software error data used in estimating model parameters. Since the AIC can be considered as an approximate distance from the true probability model, the model with smaller AIC better fits to the observed data. The degree of freedom in the model is essentially equal to the number of effective model parameters. For example, if one parameter is zero, then the parameter is not effective. Furthermore, consider the case where the model parameters have any constraint such as $p_1 + p_2 + p_3 = 1$. Then the number of model parameters is 3, but if the first two parameters are fixed, the remaining parameter can be fixed. Thus the number of effective parameters in this case is 2. In this paper, Model 1, Model 2 and Model 3 have 6 degrees, 6 degrees and 5 degrees of freedom, respectively. Although the degree of freedom in Model 3 is 5, but not 8, because the probabilities of three kinds of transitions; from the critical error to the critical error, from the critical error to the major error and from the major error to the critical error, become 0.

Table 2 presents the KS distance for the critical error in Model 1, Model 2 and Model 3 at the 38-th week. In the table, the asterisk indicates the best model in terms of the minimization of KS distance. The notation ‘n/a’ implies that the criterion cannot be computed because the

Table 2 Model comparison by the KS distance (critical)

KS distance			
NHPP	Model 1	Model 2	Model 3
Exp	0.4667*	0.5188	0.5052
Ral	0.2622*	0.4208	0.5613
Gam	0.3288*	0.4797	0.5540
LPe	n/a	–	0.5205

Table 3 Model comparison by the KS distance (major)

KS distance			
NHPP	Model 1	Model 2	Model 3
Exp	0.0971	0.1010	0.0934*
Ral	0.1202*	0.1204	0.1258
Gam	0.0947*	0.1135	0.0997
LPe	0.1070	–	0.1052

Table 4 Model comparison by the KS distance (minor)

KS distance			
NHPP	Model 1	Model 2	Model 3
Exp	0.0987*	0.1027	0.0995
Ral	0.1740	0.1738	0.1713*
Gam	0.1459	0.1394*	0.1449
LPe	0.1027	–	0.1022

Table 5 Model comparison by the MSE

MSE			
NHPP	Model 1	Model 2	Model 3
Exp	13.88	10.05*	13.98
Ral	35.25	35.13*	35.17
Gam	72.61	73.27	70.61*
LPe	n/a	–	10.53

maximum likelihood estimates take negative values. Also, the LPe cannot be applied in Model 2 and therefore we omit the result of LPe in Model 2. Similarly, Tables 3 and 4 show the KS distance for major error and minor error at the 38-th week, respectively. From these results, it can be found that the KS distance in Model 1 is relatively smaller than that in Model 2 and Model 3. In the viewpoint of better fitting to the expected number of errors, Model 1 may be superior to the other models. In Table 5, it is observed that the MSEs in Model 1 are smaller than those in Model 3. However, in this case, we can find the advantage of Model 2, that is, the MSEs in Model 2 are smallest among three models. The MSE is computed by the sum of square errors for the critical error, the major error and the minor error, so that we can evaluate the goodness-of-fit for the three kinds of errors simultaneously by the MSE. Hence it is seen that Model 2 is superior to the other models in terms of the fitness for the number of errors, by regarding for the balance of three errors.

Table 6 presents the AICs for Model 1, Model 2 and Model 3. Dissimilar to the cases of KS distance and MSE, the AICs of Model 3 are relatively smaller than those of

Table 6 Model comparison by the AIC

AIC			
NHPP	Model 1	Model 2	Model 3
Exp	321.06	321.76	317.48*
Gam	394.95	394.83	392.33*
Ral	440.82	440.46	439.81*
LPe	n/a	–	316.35

Model 1 and Model 2. This is because the AIC strongly depends on the degree of freedom. In this case, Model 1 and Model 2 have 6 parameters, but Model 3 includes less parameters than the other models, i.e., 5. Since the AIC is smaller as the number of parameters is less, the resulting AICs in Model 3 are smaller than the other cases.

4.2 Prediction performance

We investigate the prediction performance for each model. The sum of absolute errors (SAE) between the prediction for the number of software errors and the observed data is used as the criterion to measure the prediction accuracy. The SAE is defined as follows.

$$SAE = \sum_{k \in K} \sum_{i=1}^q |\hat{n}_i(k) - n_i(k)|, \quad (44)$$

where $\hat{n}(k)$ is the prediction for the number of software errors which is derived from the model and K denotes the period in which the prediction is carried out. In this paper, the prediction is performed in the period $K = [21, 38]$.

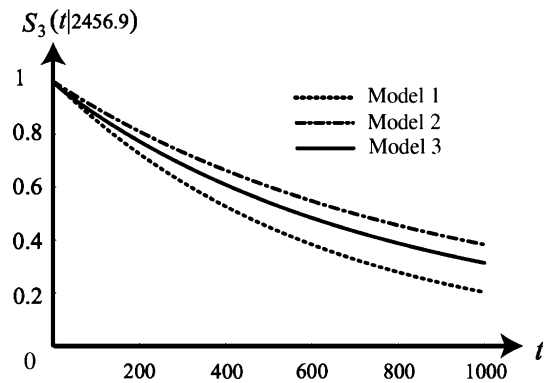
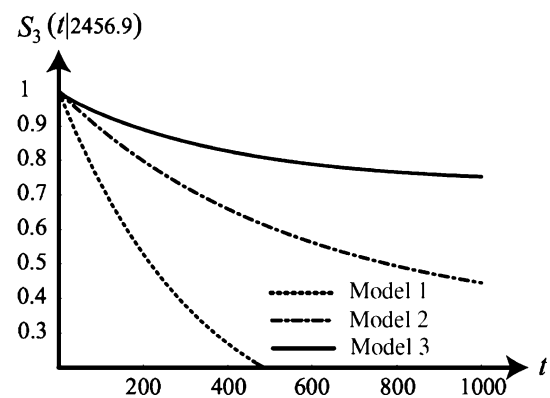
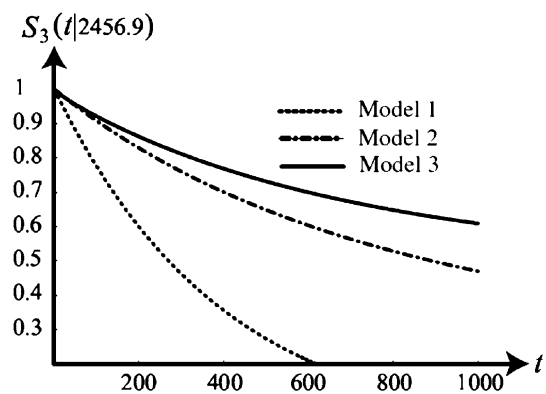
Table 7 shows the results on the prediction accuracy by the SAE. The prediction is better as the SAE is smaller. From these results, we can find that the SAEs of Model 3 are smaller than the other models, and that Model 3 is most effective in terms of the prediction of errors.

4.3 Safety assessment

Finally, we assess quantitatively the software safety based on Model 1, Model 2 and Model 3 at the 38-th week. Based on the assumption that only the critical error induces the

Table 7 Comparison of the prediction accuracy based on the SAE

SAE			
NHPP	Model 1	Model 2	Model 3
Exp	51.46	68.31	49.52*
Ral	53.85*	107.78	54.60
Gam	51.25	141.81	49.86*
LPe	n/a	–	47.74

**Fig. 1** Behavior of software safety (Exp)**Fig. 2** Behavior of software safety (Ral)**Fig. 3** Behavior of software safety (Gam)

software hazard, the software safety is defined by Eq. 21. Figures 1, 2, 3 depict the behavior of software safety for three types of NHPPs. From these results, the estimated result of software safety is different from each other, and depends on the models. The software safety based on Model 3 behaves as it is higher than the others. Thus we can conclude that Model 3 is the optimistic model in the software safety assessment.

5 Conclusion

In this paper, we have proposed a new software safety assessment model based on the subordinated Markov chain. The proposed model has represented the cumulative number of errors experienced in the testing phase and their impact on the system by using the NHPP and the DTMC, respectively. In the numerical example, using the data labeled by critical, major and minor errors, we have investigated the goodness-of-fit and the prediction performance, and further have assessed the software safety quantitatively.

As a result, we have shown that the subordinated Markov chain model is superior to the existing models in terms of the goodness-of-fit and provides the good prediction for the future occurrence errors. In future, we will consider the software release problem based on the subordinated Markov chain model, by taking account of both impact of software errors and the safety assessment.

References

- Akaike H (1973) Information theory and an extension of the maximum likelihood principle. In: Petrov BN, Csaki F (eds) 2nd International symposium on information theory, pp 267–281, Akademiai Kiado, Budapest
- Dempster AP, Laird NM, Rubin DB (1973) Maximum likelihood from incomplete data via the EM algorithm. *J Roy Stat Soc Ser B* 39:1–38
- Goel A, Okumoto K (1979) Time-dependent error-detection rate model for software reliability and other performance measures. *IEEE Trans Reliab R*-28:206–211
- Keene SJ Jr (1992) Assuring software safety. In: *Proceedings IEEE Annual Reliability and Maintainability Symposium*, pp 274–279
- Kijima M (1997) Markov processes for stochastic modeling. Chapman & Hall, London
- Kullback S (1959) Information theory and statistics. Wiley, New York
- Leveson NG (1986) Software safety: why, what, and how. *ACM Comput Surv* 18:125–163
- Lyu MR (1996) Handbook of software reliability engineering. McGraw-Hill, New York
- Misra PN (1983) Software reliability analysis. *IBM Syst J* 22: 262–270
- Musa JD (1999) Software reliability engineering. McGraw-Hill, New York
- Musa JD, Okumoto K (1984) A logarithmic poisson execution time model for software reliability measurement. In: *Proceedings of the 7th International Conference on Software Engineering*, pp 230–238, IEEE CS Press/ACM
- Ohishi K, Okamura H, Dohi T (2009) Gompertz software reliability model: estimation algorithm and empirical validation. *J Syst Softw* 82(3):535–543
- Okamura H, Dohi T (2006a) Building phase-type software reliability models. In: *Proceedings of the 17th International Symposium on Software Reliability Engineering*, pp 289–298, IEEE CS Press, Los Alamitos
- Okamura H, Dohi T (2006b) EM algorithm for discrete software reliability models: a unified parameter estimation method. *Opsearch* 42(4):355–377
- Okamura H, Dohi T (2008) Hyper-Erlang software reliability model. In: *Proceedings of the 14th Pacific Rim international symposium on dependable computing*, pp 232–239, IEEE CS Press, Los Alamitos
- Okamura H, Watanabe Y, Dohi T (2002) Estimating mixed software reliability models based on the EM algorithm. In: *Proceedings of the international symposium on empirical software engineering*, pp 69–78, IEEE CS Press, Los Alamitos
- Okamura H, Watanabe Y, Dohi T (2003) An iterative scheme for maximum likelihood estimation in software reliability modeling. In: *Proceedings of the 14th international symposium on software reliability engineering*, pp 246–256, IEEE CS Press, Los Alamitos
- Okamura H, Dohi T, Osaki S (2004) EM algorithms for logistic software reliability models. In: *Proceedings of the 22nd IASTED international conference on software engineering*, pp 263–268, ACTA Press, Calgary
- Pham H (2000) Software reliability. Springer-Verlag, Singapore
- Schick GJ, Wolverton RW (1978) An analysis of competing software reliability models. *IEEE Trans Softw Eng SE*-4:104–120
- Tokuno K, Yamada S (1999) Stochastic software safety/reliability measurement and its application. *Ann Softw Eng* 8:123–145
- Wu CFJ (1983) On the convergence properties of the EM algorithm. *Ann Stat* 11:95–103
- Yamada S, Osaki S (1985) Software reliability growth modeling: models and applications. *IEEE Trans Softw Eng SE*-11: 1431–1437